416. Partition Equal Subset Sum Medium

Acceptance: 49.1% | LeetCode

Given an integer array `nums`, return `true` *if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or* `false` *otherwise*.

Example 1:

```
**Input:** nums = [1,5,11,5]
**Output:** true
**Explanation:** The array can be partitioned as [1,
5, 5] and [11].
```

Example 2:

```
**Input:** nums = [1,2,3,5]
**Output:** false
**Explanation:** The array cannot be partitioned into
equal sum subsets.
```

Constraints:

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 100`

Claude Opus Solution

Partition Equal Subset Sum — Complete Solution

> ⚠️ **Note on reference solutions:** `solution1.py` is correct, but `solution2.py` has a bug! It checks `if total_sum % 2 == 0` (returns False when even), which is backwards. It should be `% 2 == 1`. I'll provide a correct version below.

---

1. Intuition

The **key insight** is reframing the problem:

- If we split `nums` into two subsets with equal sums, each subset must sum to `total_sum / 2`.
- **Immediate consequence:** if `total_sum` is **odd**, it's impossible (can't split an odd number into two equal halves).
- So the problem reduces to: **"Can we find a subset of `nums` that sums to exactly `total_sum / 2`?"**

This is the classic **0/1 Knapsack / Subset Sum** problem in disguise — each number is either *included* in the subset or *not*, and we want to hit an exact target.

---

2. Approach (Pseudocode)

```
1. Compute total = sum(nums)
2. If total is odd  → return False (cannot be halved)
3. Set target = total / 2
4. dp[j] = True if some subset of seen numbers sums to j
   - Initialize dp[0] = True (empty subset sums to 0)
5. For each num in nums:
       For j from target down to num:   # iterate BACKWARDS (crucial!)
           dp[j] = dp[j] OR dp[j - num]
6. Return dp[target]
```

Why iterate backwards? In 0/1 knapsack each item is used **at most once**. Updating `dp[j]` from `dp[j-num]` going *backwards* ensures we use the *previous* row's value (item not yet used), preventing the same `num` from being counted multiple times.

---

3. Python Solution

```python
from typing import List


class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        total_sum = sum(nums)

        # If the total is odd, it's impossible to split into two equal halves.
        if total_sum % 2 == 1:
            return False
```

```
        # Each subset must sum to exactly half of the total.
        half_sum = total_sum // 2

        # dp[j] = True if some subset of processed numbers sums to j.
        # dp[0] = True because the empty subset always sums to 0.
        dp = [True] + [False] * half_sum

        # Process each number — decide to include it or not.
        for num in nums:
            # Iterate BACKWARDS so each num is used at most once (0/1 knapsack).
            # We stop at 'num' because dp[j-num] requires j >= num.
            for j in range(half_sum, num - 1, -1):
                # dp[j] becomes True if it was already True,
                # OR if we can reach (j - num) and then add 'num'.
                dp[j] |= dp[j - num]

            # Optional early exit: target reached, no need to continue.
            if dp[half_sum]:
                return True

        return dp[half_sum]
```

Alternative: Bitwise Trick (very fast in Python)

```
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        total_sum = sum(nums)

        # Odd total can never be split evenly.
        if total_sum % 2 == 1:          # <-- correct condition (== 1, not == 0)
            return False

        half_sum = total_sum // 2

        # 'bits' is a big integer used as a bitmask.
        # Bit i is set <=> some subset can sum to i.
        # Start with bit 0 set (empty subset sums to 0).
        bits = 1
        for num in nums:
            # Shifting left by 'num' adds 'num' to every reachable sum.
            # OR-ing keeps all old reachable sums too.
            bits |= bits << num

        # Check if the bit at position 'half_sum' is set.
        return (bits >> half_sum) & 1 == 1
```

The bitwise approach leverages Python's arbitrary-precision integers to perform all DP transitions in parallel — typically the fastest in practice.

---

4. Complexity Analysis

Let `n = len(nums)` and `m = sum(nums)` (so `target = m/2`).

Approach	Time	Space	Explanation
DP Tabulation	`O(n · m)`	`O(m)`	Outer loop over `n` items, inner loop over `~m/2` sums. Single 1-D array of size `m/2`.
Bitwise	`O(n · m / 64)`	`O(m / 64)`	Same asymptotic `O(n·m)` but bit operations process 64 bits per word, giving a large constant-factor speedup.

For the given constraints (`n ≤ 200`, `nums[i] ≤ 100` ⟹ `m ≤ 20000`), both run comfortably fast.

---

5. Alternative Approaches

1. **Recursion + Memoization (Top-Down DP)** ```python from functools import lru_cache def canPartition(self, nums): total = sum(nums) if total % 2: return False target = total // 2

@lru_cache(None) def dfs(i, remaining): if remaining == 0: return True if remaining < 0 or i == len(nums): return False # Try including nums[i] OR skipping it. return dfs(i + 1, remaining - nums[i]) or dfs(i + 1, remaining) return dfs(0, target) `` `Same` O(n · m)` complexity but uses recursion stack + memo table — more intuitive but higher overhead.

2. **2-D DP Table** — `dp[i][j]` = can first `i` items reach sum `j`. Clearer to reason about (`O(n·m)` space) but the 1-D rolling array (our main solution) is strictly better in space.
3. **Pure Backtracking (no memo)** — exponential `O(2^n)`; only viable for tiny inputs.

Key Patterns to Remember

- **"Partition into equal halves"** → **Subset Sum to `total/2`**.
- **Subset Sum / 0-1 Knapsack** → 1-D DP with a **backwards** inner loop.
- **Parity check first** (odd sum ⟹ instant `False`) is a cheap, powerful pruning step.

Codex Solution

1. Intuition

To split `nums` into two subsets with equal sum, the total sum must be even.

If:

```
total = sum(nums)
```

then each subset must sum to:

```
target = total / 2
```

So the problem becomes:

> Can we pick some numbers from `nums` whose sum is exactly `target`?

This is the classic **0/1 Knapsack / Subset Sum** pattern.

Each number can be used either:

1. Not used
2. Used once

We can use dynamic programming to track which subset sums are possible.

---

2. Approach

Use a 1D boolean DP array:

```
dp[s] = True if we can form sum s using some processed numbers
```

Initially:

```
dp[0] = True
```

Because sum `0` is always possible by choosing nothing.

For each number `num`, update possible sums from right to left:

```
for s from target down to num:
    dp[s] = dp[s] or dp[s - num]
```

Why right to left?

Because each number can only be used once.
Going right to left prevents the current number from being reused multiple times in the same iteration.

Pseudocode

```
total = sum(nums)

if total is odd:
    return False
```

```
target = total / 2

dp = boolean array of size target + 1
dp[0] = True

for num in nums:
    for s from target down to num:
        if dp[s - num] is True:
            dp[s] = True

return dp[target]
```

3. Python Solution

```python
from typing import List


class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        # Calculate total sum of all numbers
        total_sum = sum(nums)

        # If total sum is odd, it cannot be split into two equal integer sums
        if total_sum % 2 == 1:
            return False

        # Each subset must sum to exactly half of total_sum
        target = total_sum // 2

        # dp[s] means:
        # Can we form sum 's' using some subset of the numbers processed so far?
        dp = [False] * (target + 1)

        # Sum 0 is always possible: choose no elements
        dp[0] = True

        # Process each number once
        for num in nums:
            # Traverse backwards to avoid using the same number multiple times
            #
            # Example:
            # If num = 3 and we loop forward,
            # dp[3] could become True, then dp[6] could use that same 3 again.
            #
            # Backward iteration ensures each num is only used once.
            for curr_sum in range(target, num - 1, -1):
                # If we could previously make curr_sum - num,
                # then by adding num, we can now make curr_sum.
                if dp[curr_sum - num]:
                    dp[curr_sum] = True

            # Optional early exit:
            # If target is already possible, no need to continue.
            if dp[target]:
                return True

        # Final answer: whether target sum can be formed
        return dp[target]
```

4. Complexity Analysis

Let:

```
n = len(nums)
target = sum(nums) / 2
```

Time Complexity

```
O(n * target)
```

For each number, we iterate through sums up to `target`.

Given constraints:

```
nums.length <= 200
nums[i] <= 100
```

The maximum total sum is:

```
200 * 100 = 20000
```

So the maximum target is:

```
10000
```

This is efficient enough.

Space Complexity

```
O(target)
```

We only store a 1D DP array of size `target + 1`.

---

5. Alternative Approaches

Approach 1: 2D Dynamic Programming

Define:

```
dp[i][s] = True if we can form sum s using the first i numbers
```

Transition:

```
dp[i][s] = dp[i - 1][s] or dp[i - 1][s - nums[i]]
```

This is easier to understand, but uses more memory.

Time complexity:

```
O(n * target)
```

Space complexity:

```
O(n * target)
```

The 1D DP solution is just a space-optimized version of this.

Approach 2: Bitset DP

We can represent possible sums using bits.

If bit `s` is set, then sum `s` is possible.

For each number `num`:

```
bits |= bits << num
```

This shifts all existing possible sums by `num`, meaning we include `num`.

Example implementation:

```python
from typing import List


class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        total_sum = sum(nums)

        if total_sum % 2 == 1:
            return False

        target = total_sum // 2
```

```
        # Bit 0 is set, meaning sum 0 is possible
        bits = 1

        for num in nums:
            # Add num to every previously possible sum
            bits |= bits << num

        # Check if target-th bit is set
        return (bits & (1 << target)) != 0
```

This is often very fast in Python because integer bit operations are implemented efficiently.

Conceptually, its complexity is still approximately:

```
O(n * target)
```

but with a much smaller constant factor.